

Appunti di Artificial Intelligence from Engineering to Arts

Laura Straccini

April 23, 2026

Contents

1	Introduzione	1
1.1	Introduzione al corso	1
2	Computational Intelligence	3
2.1	Computational Intelligence nel mondo dei dati	3
2.1.1	Evoluzione della CI	3
2.1.2	Modelli Intelligenti vs. Modelli Algoritmici	4
2.1.3	Apprendimento e Adattamento ai Dati	5
2.1.4	L'importanza della Generalizzazione	6
2.2	Soft Computing e Calcolo Bio-Ispirato	6
2.2.1	Algoritmi bio-ispirati	7
2.3	Swarm intelligence as State Equations	10
2.4	Fuzzy logic	11
2.4.1	Concetto di insieme	12
2.4.2	Fuzzy systems	13
3	Reti Neurali	15
3.1	Neural Network Concepts and Paradigms	15
3.1.1	Age of Camelot	15
3.1.2	Dark Age	19
3.1.3	The Renaissance	20
3.2	Reti Neurali Non Supervisionati	21
3.2.1	Architettura Schematica di KNN SOM	22
3.2.2	Kohonen Learning Algorithm	24
3.3	RNN - Recurrent Neural Networks	24
3.3.1	LSTM - Long Short-Term Memory	24
3.4	Transformer	25
3.4.1	Self Attention	26
3.4.2	Multi-Head Attention	27
3.4.3	Masked Multi-head Attention	28
3.4.4	Transformer e musica	29

<i>CONTENTS</i>	ii
4 Reti Generative	30
4.1 Reti Generative Avversarie (GAN)	30
4.1.1 Sintesi Visiva	32

Chapter 1

Introduzione

1.1 Introduzione al corso

La prima parte che trattiamo è quella più ingegneristica, un primo svincolo che si adopera dai metodi computazionali classici che sono rigorosi. Laddove abbiamo gli strumenti adonei per ottenere ottimi risultati usiamo questi. La natura dei problemi può essere non lineare per cui il calcolo rigoroso è troppo dispendioso o impossibile da applicare. Si tentano strade alternative (quando la risposta rigorosa è nota per aver un criterio di valutazione).

La fonte primaria ispiratrice della computational intelligence. Nasce come proposta da un iraniano Zadeh, che propone la logica fuzzy, in cui diceva ottimi risultati potevano essere ottenuti da singole applicazioni.

Il corso si concentra sulla parte più legata a come adoperare in alcuni contesti un tipo di ia piuttosto che un altro.

Tecniche di pre processing dei dati. La flessibilità di ia e come adoperarla in contesti diversi. Tutte le varie tecniche vengono adoperate e apprezzate nei vari contesti.

Faremo delle simulazioni con Python. Chiameremo l'ia e gli chiederemo di fare cose, in modo che ci restituisce un risultato.

Vedremo come trasformare un segnale musicale in un'immagine.

Fuzzy logic, logica come alternativa alla logica booleana, che è basata su true e false.

L'applicazione che faremo prevalentemente è sul segnale musicale, posso usare protocolli informatici (MIDI Musical Instrument Digital Interface).

Posso dire di spostare il calcolo da uno analitico chiuso a uno soft infatti si chiamava soft computing. Con il passare del tempo tutte le tecniche di natura biologica e naturale sono state inglobate in questo paradigma, che è diventato più ampio.

Quali problemi trattano questi algoritmi evolutivi? Problemi di ottimizzazione o problemi inversi. Il problema di ottimizzazione significa che è gestito

da equazioni governate da alcuni parametro vorrei sapere la configurazione ottima di quei parametri per ottenere una configurazione ottima.

I problemi inversi sono quelli in cui abbiamo un certo numero di dati e vorremmo sapere quale è la configurazione che ha generato quei dati, o che è più vicina a quei dati. Ho un modello matematico che mi da per esempio le previsioni del tempo, questo modello non è stato attualizzato ai dati che servono a me. All'interno a quelle equazioni ci sono dei parametri che sono stati messi in modo generico ma che dovrebbero essere attualizzati. Per esempio il modello di una parabola:

$$y = ax^2 + bx + c$$

Con a , b , c generici ho il modello, poi piego il modello al caso che mi interessa, ovvero cambio i valori di a , b , c in modo da far sì che la parabola approssimi al meglio i dati osservati.

Altri metodi (es. simulated annealing) sono più adatti a problemi di ottimizzazione (esempio lo usano molto i fisici), altri (es. reti neurali) sono più adatti a problemi inversi, ma non è detto che non possano essere usati anche per l'altro tipo di problema.

Tutti questi algoritmi sono stocastici, c'è una componente stocastica in modo da aumentare la capacità di questi algoritmi di performare bene. Se uso una determinata funzione per trovare per esempio il minimo di una funzione, potrei essere bloccato in un minimo locale, se invece uso una componente stocastica, posso uscire da questo minimo locale e trovare un minimo globale.

Questi algoritmi vanno visti in funzione delle loro capacità, quindi vanno testati. Quale algoritmo va bene per piccoli spazi e quale per grandi spazi.

Le più grandi categorie con cui vengono classificate sono: grado di esplorazione (exploration) e grado di sfruttamento (exploitation).

La rete neurale è difatto un problema di ottimizzazione. La rete neurale dovrebbe emulare il cervello di un vertebrato, ci sono dei dati che vengono presi dall'esterno (banca dati). Per esempio ho in input $y = f(x)$, poi prendo questo dati che passa per il neurone di ingresso faccio una sfilza di nodi (chiamati neuroni nascosti), l'informazione di input viene veicolata dal neurone di ingresso a tutti i neuroni nascosti, viene veicolata in modo tale da avere dei pesi diversi, dando luogo a un uscita, queste uscite vengono dati in pasto ad altri neuroni di un altro strato nascosto (perché non è nè input nè output). Ciascuno di questi passaggi ha un altro peso. Tutta questa informazione viene poi convogliata verso il neurone d'uscita (y). Quindi tutta la rete è $f(x)$ e in ingresso ho x .

Allora provo a mettere i valori sperimentali, all'inizio i pesi sono a caso. Una volta che ho tutti i risultati a caso e so quelli che dovrebbero uscire posso applicare un algoritmo di ottimizzazione per minimizzare l'errore. Una volta applicato l'algoritmo riinserisco i valori e itero finché non ottengo i risultati aspettati.

L'insieme dei pesi dei valori sinapsici è lo spazio delle soluzioni.

Reti neurali con memoria (recurrente neural network). Reti neurali generative.

Chapter 2

Computational Intelligence

2.1 Computational Intelligence nel mondo dei dati

La **Computational Intelligence (CI)**, o intelligenza computazionale, è un ramo dell'Intelligenza Artificiale che si concentra sullo sviluppo di metodi computazionali in grado di apprendere, adattarsi, gestire l'incertezza e funzionare in modo efficace in ambienti molto complessi. Questi metodi ottengono enormi successi traendo ispirazione diretta dall'intelligenza biologica e dal mondo naturale, creando quelli che vengono definiti "sistemi intelligenti".

Questi algoritmi intelligenti includono:

- reti neurali artificiali
- calcolo evolutivo
- intelligenza dello sciame
- sistemi immunitari artificiali
- sistemi fuzzy

La fonte primaria ispiratrice della computational intelligence. Nasce come proposta da un iraniano Zadeh, che propone la logica fuzzy, in cui diceva ottimi risultati potevano essere ottenuti da singole applicazioni.

2.1.1 Evoluzione della CI

Nel mondo dei dati, la CI rappresenta un pilastro fondamentale per quattro motivi principali:

1. **Gestione dell'incertezza:** Fornisce gli strumenti per elaborare dati ambigui, rumorosi o incompleti, permettendo di prendere decisioni e fare previsioni anche quando le informazioni non sono perfette.

2. **Apprendimento e adattamento:** Le tecniche di CI estraggono informazioni direttamente dai dati per adattare i propri parametri. Questo è cruciale in un mondo dinamico dove i dati cambiano rapidamente nel tempo
3. **Complessità:** Riesce a gestire e risolvere problemi altamente complessi che risultano intrattabili con i metodi analitici e ingegneristici tradizionali.
4. **Automazione:** Automatizza processi e decisioni complesse, liberando l'essere umano e permettendogli di concentrarsi su attività più creative e strategiche.

2.1.2 Modelli Intelligenti vs. Modelli Algoritmici

La vera rivoluzione della CI nel campo dei dati è il passaggio dai modelli algoritmici a quelli intelligenti.

Modelli Algoritmici

Questi modelli rappresentano l'approccio computazionale classico:

- **Logica e Sequenze:** Seguono una sequenza logica di istruzioni predefinite. Si basano su algoritmi rigorosi, che definiscono passo dopo passo le operazioni necessarie per risolvere un problema.
- **Determinismo:** Sono strettamente deterministici. Questo significa che, operando nelle medesime condizioni, produrranno sempre risultati identici a fronte di uno stesso input.
- **Limitati alla Programmazione:** Sono vincolati in modo stringente al codice scritto dal programmatore. Per eseguire un determinato compito, richiedono che la sequenza di istruzioni sia inserita esplicitamente e dettagliatamente all'interno del programma.

Modelli Intelligenti Questi modelli, tipici della Computational Intelligence, superano i limiti della programmazione rigida:

- **Apprendimento e Adattamento:** La loro caratteristica principale è la capacità di imparare dai dati e di adattarsi alle nuove informazioni, il tutto senza essere esplicitamente programmati per farlo.
- **Gestione dell'Incertezza:** Mentre i modelli algoritmici richiedono dati precisi, i modelli intelligenti possono operare in ambienti complessi gestendo l'incertezza. Sono in grado di prendere decisioni e fare previsioni anche in presenza di dati rumorosi o incompleti.
- **Capacità Cognitive:** Riescono ad emulare alcune delle capacità cognitive umane, come l'adattamento alle variazioni dell'ambiente, l'apprendimento continuo e il ragionamento.

2.1.3 Apprendimento e Adattamento ai Dati

Nell'ambito della Computational Intelligence (CI), l'apprendimento e l'adattamento sono concetti strettamente legati che traggono una forte ispirazione dai processi del mondo biologico e naturale.

Apprendimento (Inspirazione Biologica e Artificiale) Nel mondo biologico, gli organismi viventi imparano continuamente dall'esperienza, come ad esempio un animale che impara a evitare un determinato cibo dopo averne provato gli effetti nocivi. Questo principio è stato traslato nell'Intelligenza Artificiale, trovando la sua massima espressione nelle reti neurali artificiali. In queste architetture, i "neuroni" (le unità di elaborazione) sono in grado di modificare la forza delle proprie connessioni in base ai dati di input che ricevono, consentendo alla rete di accumulare esperienza e "apprendere" nel tempo.

Adattamento e Adattamento Dinamico In biologia, l'adattamento è il processo evolutivo attraverso cui una specie diventa progressivamente più idonea a sopravvivere nel suo ambiente (come gli uccelli che sviluppano becchi più adatti a raccogliere cibo difficile da raggiungere). Nei sistemi di CI, l'adattamento viene implementato tramite algoritmi che modificano il comportamento del sistema in base all'ambiente o ai nuovi dati in ingresso, migliorandone le prestazioni nel tempo.

Un concetto fondamentale in questo campo è l'**adattamento dinamico**, ovvero la capacità di un sistema di modificare i suoi parametri in tempo reale mentre è in esecuzione in un ambiente mutevole.

Questo è un enorme vantaggio perché permette al sistema di adattarsi istantaneamente senza aver bisogno di essere fermato per un "re-training" (ri-addestramento) offline. I sistemi dotati di processi adattivi riducono drasticamente il tempo necessario per giungere a una soluzione rispetto ai vecchi metodi che dovevano calcolare o esplorare gran parte delle possibilità.

- **Apprendimento Supervisionato:** L'algoritmo viene addestrato fornendogli un set di dati in cui le risposte corrette (output desiderati) sono già note. Il sistema si adatta calcolando l'errore rispetto all'output corretto e applicando le necessarie correzioni. È molto utilizzato per problemi di classificazione e regressione.
- **Apprendimento Non Supervisionato:** Al sistema vengono forniti dati "grezzi", privi di etichette o output desiderati. L'algoritmo deve adattarsi esplorando i dati senza una guida esterna, con lo scopo di identificare autonomamente pattern, regole o strutture nascoste (utilizzato molto nel clustering o nella riduzione dimensionale).
- **Apprendimento per Rinforzo:** Il sistema impara agendo all'interno di un ambiente dinamico. L'algoritmo si adatta in base alle conseguenze

delle proprie azioni, ricevendo un feedback sotto forma di ricompensa (se l'azione è giusta) o penalità (se è sbagliata).

2.1.4 L'importanza della Generalizzazione

Un concetto cardine quando si applica la CI ai dati è la **generalizzazione**. Un modello non deve limitarsi a memorizzare i dati di addestramento (fenomeno noto come *overfitting*), ma deve riuscire a estrarre regole e pattern profondi per applicarli con successo a nuovi dati mai visti prima. La generalizzazione è cruciale per l'efficienza del sistema, per la sua capacità di affrontare la variabilità e il rumore del mondo reale, e per sapersi adattare a contesti differenti. L'abilità di generalizzare viene solitamente misurata testando il modello su un set di dati separato da quello di addestramento.

2.2 Soft Computing e Calcolo Bio-Ispirato

Il **Soft Computing** è un campo strettamente legato all'Intelligenza Computazionale e definisce una forma di **computazione bio-ispirata**. Bio-ispirata principalmente per algoritmi genetici e reti neurali. Nella logica non c'è sempre una risposta univoca, per esempio colore dei capelli, sono bianchi ma ci sono anche bianchi sporchi, bianchi con riflessi, bianchi con sfumature, ecc.

Si basa su algoritmi euristici che traggono la loro ispirazione direttamente dal mondo naturale, seguendo un approccio che viene anche definito "ottimizzazione naturale" o degli "algoritmi bio-ispirati".

L'obiettivo di questi algoritmi è replicare o adattarsi ai processi osservati in natura, imitando il comportamento degli organismi o dei sistemi biologici. Questo tipo di computazione si rivela particolarmente efficace per risolvere problemi molto complessi o per effettuare ottimizzazioni in contesti in cui le soluzioni devono sapersi adattare a cambiamenti dinamici, esattamente come le specie in natura si adattano al proprio ambiente.

I principali metodi e algoritmi che rientrano sotto l'ombrello del Soft Computing includono:

- **Reti Neurali Artificiali:** Ispirate alla struttura e al funzionamento del cervello umano, sono composte da unità di elaborazione chiamate "neuroni", collegate tra loro in schemi complessi, che collaborano per apprendere dai dati e fare previsioni o prendere decisioni.
- **Algoritmi Genetici:** Basati sulla teoria dell'evoluzione di Darwin, permettono di affrontare problemi di ottimizzazione di altissima complessità che richiederebbero tempi impraticabili con gli algoritmi tradizionali. Utilizzano concetti come la selezione naturale del "più adatto", il crossover

(incrocio genetico) e la mutazione per generare una popolazione di possibili soluzioni e farla evolvere progressivamente nel tempo.

- **Logica Fuzzy:** È un'estensione della logica classica che permette di superare la rigidità degli insiemi binari (dove un valore è strettamente "vero" o "falso") introducendo i "gradi di verità". La logica fuzzy permette di trattare concetti sfumati, imprecisi o vaghi (come "quasi vero" o "parzialmente falso"), consentendo ai sistemi di modellare molto meglio l'ambiguità e la complessità del mondo reale.
- **Algoritmi euristici ispirati a comportamenti collettivi (Swarm Intelligence):** Sebbene raggruppati tra gli algoritmi naturali, spiccano modelli come la Particle Swarm Optimization (PSO) e l'Ant Colony Optimization (ACO). La PSO replica il comportamento sociale degli stormi di uccelli o dei banchi di pesci, i cui membri comunicano e condividono le proprie "scoperte" per guidare l'intero gruppo verso l'obiettivo migliore. L'ACO, invece, si ispira al comportamento delle formiche, che per risolvere problemi combinatori lasciano tracce chimiche (feromoni) sul percorso per influenzare le decisioni del resto della colonia. Sono utilizzati per risolvere problemi di ottimizzazione dei percorsi,

2.2.1 Algoritmi bio-ispirati

Problemi di ottimizzazione Mettiamo che dobbiamo affrontare un problema di ottimizzazione, ovvero trovare quei parametri che in uno spazio di soluzioni mi minimizzano o massimizzano una funzione obiettivo. Qualora scrivo la ricerca del funzione la ricerca del funzionale per massimizzarlo, quel funzionale si chiama fitness function, e il processo di ottimizzazione è chiamato processo di evoluzione. Se devo minimizzare la chiamerò funzione costo.

Massimizzare questa funzione significa trovare delle caratteristiche indicative del problema, Determinare quali sono i parametri che mi permettono di massimizzare la funzione obiettivo, e quindi trovare la soluzione migliore al problema che sto affrontando.

Si instaura un processo iterativo in cui, a ogni iterazione, vengono generate nuove soluzioni (o "individui") che vengono valutate in base alla loro "fitness" (cioè quanto bene soddisfano la funzione obiettivo). Le soluzioni migliori vengono selezionate per "riprodursi" e generare nuove soluzioni, che a loro volta vengono valutate e selezionate, e così via, fino a quando non si raggiunge una soluzione soddisfacente o si esaurisce il tempo disponibile.

Mettiamo che dobbiamo cercare un minimo globale di una funzione complessa, con molti minimi locali. Un algoritmo genetico, grazie alla sua capacità di esplorare ampi spazi di soluzioni e di evitare di rimanere intrappolato in minimi locali, può essere molto più efficace di un algoritmo tradizionale di ottimizzazione, che potrebbe facilmente rimanere bloccato in un minimo locale

senza raggiungere il minimo globale.

Pure gli algoritmi bio-ispirati dipendono da dei parametri, come la dimensione della popolazione, il tasso di mutazione, il tasso di crossover, ecc. Li posso rendere più o meno esplorativi a seconda di come li setto, e questo è un aspetto molto importante perché mi permette di bilanciare l'esplorazione (la capacità di cercare nuove soluzioni) e lo sfruttamento (la capacità di migliorare le soluzioni già trovate).

A volte quando la mole di parametri è molto grande si cerca di privilegiare algoritmi meno precisi ma più veloci, per esempio la ricerca del gradiente.

Un'altra importante classificazione dei problemi di ottimizzazione

Le euristiche (come gli Algoritmi Genetici, la Particle Swarn Optimization, ecc.) vengono suddivise in base a come bilanciano esplorazione e sfruttamento rispetto alle dimensioni dello spazio delle soluzioni:

- **Euristiche per VHDSS (Very High Dimension Solution Space):** Quando il problema è a dimensioni elevatissime e l'utente non ha alcuna idea di dove si trovi la soluzione, è necessario utilizzare algoritmi ad alta esplorazione. In questi casi, il confronto premia il Calcolo Evolutivo (come gli Algoritmi Genetici) e in generale i metodi di Swarm Intelligence, capaci di mappare ampie porzioni di spazio sconosciuto.
- **Euristiche per HDSS (High Dimension Solution Space):** Quando si ha solo un sospetto sulla zona in cui risiede la soluzione, le euristiche più performanti per l'esplorazione includono algoritmi come la Bacterial Chemotaxis, il Simulated Annealing, la Tabu Search e l'uso della Logica Fuzzy.
- **Euristiche per LDSS (Low Dimension Solution Space):** Quando si ha già un'idea abbastanza precisa delle coordinate ottimali, la fase di esplorazione diventa secondaria rispetto a quella di sfruttamento (exploitation). In questa specifica fase di affinamento, il confronto premia algoritmi come la Particle Swarm Optimization (PSO) e la Flock of Starlings Optimization.

Explanability (spiegabilità) è un aspetto cruciale in questo contesto, soprattutto quando si utilizzano modelli intelligenti che operano in ambienti complessi e dinamici. La capacità di spiegare le decisioni e i processi di apprendimento di un sistema di CI è fondamentale per garantire la fiducia degli utenti, facilitare la diagnosi di errori e migliorare la trasparenza complessiva del sistema. Un sistema di CI con una buona explainability è in grado di fornire spiegazioni chiare e comprensibili sulle ragioni alla base delle sue decisioni, sui dati che ha utilizzato per apprendere e sulle strategie che ha adottato per adattarsi a nuove situazioni.

Consideriamo l'euristica Particle Swarm Optimization (PSO) come esempio di algoritmo bio-ispirato. Questi algoritmi hanno una ispirazione di intelligenza di sciame (ma anche sociale) per questo a lavorarci sopra c'erano anche sociologi, psicologi, biologi, ecc. Ciascun individuo che farà parte dello sciame, verrà inizializzato nello spazio delle soluzioni con una posizione e una velocità casuali. In base al punto in cui si trova ogni individuo misurerà la sua fitness, e in base alla fitness degli altri individui, si muoverà verso la posizione migliore che ha trovato lui stesso e verso la posizione migliore trovata dagli altri individui. In questo modo, lo sciame si muove collettivamente verso le aree dello spazio delle soluzioni che presentano una fitness più elevata, esplorando e sfruttando le informazioni condivise tra i membri dello sciame per trovare soluzioni ottimali o quasi ottimali.

Gli stimoli sono:

- **La mia esperienza personale** e quindi l'informazione che ho accumulato fino a quel momento (la mia posizione nello spazio delle soluzioni e la mia fitness)
- **L'esperienza sociale**, ovvero l'informazione che condividono gli altri membri dello sciame
- **L'inerzia**, ovvero la tendenza a mantenere la propria velocità e direzione attuale, che aiuta a bilanciare l'esplorazione e lo sfruttamento.

Sarà dentro un ciclo for questo:

$$v_k^j(t+1) = v_k^j(t) + \lambda^j(pbest_k^j - x_k^j(t)) + \gamma^j(gbest^j - x_k^j(t))$$

k vuol dire che lo spazio è n dimensionale, j vuol dire che ci sono m individui nello sciame, $v_k^j(t)$ è la velocità dell'individuo j nella dimensione k al tempo t, $pbest$ è la posizione migliore che ha trovato lui stesso, $gbest$ è la posizione migliore trovata dagli altri membri dello sciame, λ e γ sono dei parametri che bilanciano l'importanza di questi due stimoli.

$$V_i^{k+1} = \omega V_i^k + c_1 rand_1(...)x(pbest_i^k - s_i^k) + c_2 rand_2(...)x(gbest^k - s_i^k)...$$

In questo modo, ogni individuo si muove verso la posizione migliore che ha trovato lui stesso e verso la posizione migliore trovata dagli altri membri dello sciame, bilanciando l'esplorazione e lo sfruttamento attraverso i parametri λ e γ .

I parametri rand1 e rand2 sono dei numeri casuali che vengono generati ad ogni iterazione, e che introducono una componente di casualità nel processo di aggiornamento della velocità, che aiuta a evitare che lo sciame si blocchi in minimi locali e a favorire l'esplorazione di nuove aree dello spazio delle soluzioni.

Diagramma di flusso che rappresenta l'Algoritmo Generale PSO:

2.3 Swarn intelligence as State Equations

É una variazione importante perché si passa da un algoritmo stocastico a uno deterministico, rimandando nell'ambito della swarm intelligence, ma con un approccio più matematico e formale. Modo di approcciare in maniera supervisionato invece di lasciare la supervisione ai parametri stocastici.

Questa variazione é una variazione che la comunità scientifica del calcolo evolutivo digerisce e non digerisce, perché la natura stessa del calcolo é sfumato, quindi qualsiasi introduzione di rigidità e determinismo viene vista come una forzatura. Questi algoritmi spesso fanno riferimento a fenomeni naturali ma non li trattano come dovrebbero essere trattati.

Siamo abituati a vederer l'espressione:

$$v_k^j(t+1) = \omega v_k^j(t) + \lambda(p_{best_k}^j(t) - x_k^j(t)) + \gamma(g_{best}(t) - x_k^j(t)) + \sum_{m=1}^N h_{km} v_m^j$$

In questo modo, ogni individuo si muove verso la posizione migliore che ha trovato lui stesso e verso la posizione migliore trovata dagli altri membri dello sciame, bilanciando l'esplorazione e lo sfruttamento attraverso i parametri λ e γ , e tenendo conto anche dell'influenza degli altri individui dello sciame attraverso il termine $\sum_{m=1}^N h_{km} v_m^j$.

Il parametro h_{γ} é il parametro sociale, g_{best} (global best) é la posizione migliore trovata dall'intero sciame quindi non dovrebbe avere j all'apice.

Una volta aggiornata la velocità aggiorniamo la posizione:

$$x_k^j(t+1) = x_k^j(t) + v_k^j(t+1)$$

Peró sto sommando una velocità a una posizione, quindi non é propriamente corretto.

Per convertire questo algoritmo in un sistema dinamico, dobbiamo esprimere l'aggiornamento della velocità e della posizione in termini di equazioni differenziali o differenze discrete.

$$v_k^j(t + \Delta t) = \omega v_k^j(t) + \lambda(p_{best_k}^j(t) - x_k^j(t)) + \gamma(g_{best}(t) - x_k^j(t)) + \sum_{m=1}^N h_{km} v_m^j$$

Nel momento in cui divido per Δt ottengo:

$$\frac{v_k^j(t + \Delta t) - v_k^j(t)}{\Delta t} = \omega \frac{v_k^j(t)}{\Delta t} + \lambda \frac{p_{best,k}(t) - x_k^j(t)}{\Delta t} + \gamma \frac{g_{best}(t) - x_k^j(t)}{\Delta t} + \sum_{m=1}^N h_{km} \frac{v_m^j}{\Delta t}$$

Nel limite in cui Δt tende a zero, otteniamo un sistema di equazioni differenziali ordinarie che descrivono l'evoluzione della velocità e della posizione degli individui dello sciame nel tempo.

La forzante ($F_k(t)$).

I vantaggi di usare le forme chiuse invece di un algoritmo stocastico sono molteplici:

- **Analisi Matematica:** Le forme chiuse permettono di applicare strumenti di analisi matematica per studiare la stabilità, la convergenza e il comportamento dinamico del sistema, cosa che è molto più difficile da fare con algoritmi stocastici.
- **Controllo e Ottimizzazione:** Con un modello deterministico, è possibile progettare strategie di controllo e ottimizzazione più precise, poiché si ha una comprensione più chiara di come le variabili influenzano il sistema.
- **Prevedibilità:** I sistemi deterministici offrono una maggiore prevedibilità rispetto agli algoritmi stocastici, che possono produrre risultati diversi a ogni esecuzione a causa della loro natura casuale.
- **Efficienza Computazionale:** In alcuni casi, le forme chiuse possono essere più efficienti dal punto di vista computazionale, poiché non richiedono la generazione di numeri casuali o l'esecuzione di molte iterazioni per raggiungere una soluzione soddisfacente.
- **Applicabilità a Problemi Specifici:** In alcuni contesti, come il controllo di sistemi dinamici o l'ottimizzazione in tempo reale, un modello deterministico può essere più adatto e fornire risultati più affidabili rispetto a un algoritmo stocastico.

2.4 Fuzzy logic

Fuzzy significa rozzo, per distinguere dalla logica booleana che è pulita e precisa. C'è il ragionamento approssimato, rappresenta un modo di riprodurre in modo artificiale i processi di ragionamento umano, che spesso non sono precisi e rigidi, ma piuttosto sfumati e imprecisi.

Non è la totale negazione della logica booleana, ma una sua estensione che permette di trattare concetti sfumati e imprecisi. Alcuni concetti come caldo o freddo, alto o basso, sono intrinsecamente sfumati e non possono essere rappresentati in modo preciso con la logica booleana.

Per esempio per il caldo o freddo passiamo da una rappresentazione numerica a una qualificazione linguistica.

Concetti fondamentali della logica fuzzy:

- Modellare l'incertezza del linguaggio naturale
- Trattare il concetto di parziale verità
- Definire i valori compresi tra 0 e 1, ovvero tra il "completamente falso" e il "completamente vero"

Autore della fuzzy logic é Lotfi Zadeh, che ha introdotto il concetto di "fuzzy set" (insieme sfumato) per rappresentare insiemi che non hanno confini netti, ma piuttosto gradi di appartenenza.

2.4.1 Concetto di insieme

Secondo la logica classica, un elemento o appartiene a un insieme o non appartiene, mentre nella logica fuzzy, un elemento può appartenere a un insieme con un certo grado di appartenenza, che è rappresentato da un numero compreso tra 0 e 1.

Potremmo dire che fuzzy é una finestratura del booleano, ma in realtà un elemento può appartenere in parte anche in due insiemi opposti, per esempio un individuo può essere alto e basso allo stesso tempo, con gradi di appartenenza diversi.

L'insieme dato dalla logica classica si chiama "crisp set" (insieme nitido), mentre l'insieme dato dalla logica fuzzy si chiama "fuzzy set" (insieme sfumato).

Diagramma di Venn é l'operazione di intersezione tra insiemi crisp.

Zadeh ha esteso il concetto di insieme a quello di insieme fuzzy, introducendo il concetto di "membership function" (funzione di appartenenza), che assegna a ogni elemento un grado di appartenenza all'insieme fuzzy, rappresentato da un numero compreso tra 0 e 1.

Se un elemento ha un grado di appartenenza di 0, significa che non appartiene affatto all'insieme, mentre se ha un grado di appartenenza di 1, significa che appartiene completamente all'insieme, se é tra 0 e 1 appartiene parzialmente.

Le funzioni di Membership possono assumere diverse forme, come ad esempio funzioni triangolari, trapezoidali o gaussiane, a seconda del contesto e delle esigenze specifiche dell'applicazione.

Le funzioni di appartenenza possono essere ricondotte a 3 fattori principali:

- altezza: ti da l'estremo superiore
- core: ti da l'estremo inferiore
- supporto: ti da l'intervallo di valori per cui la funzione di appartenenza è non nulla

Un numero fuzzy non rappresenta un valore preciso, ma piuttosto un intervallo di valori con gradi di appartenenza diversi. Fuzzy singleton é un numero fuzzy che rappresenta un singolo valore con un grado di appartenenza di 1, mentre tutti gli altri valori hanno un grado di appartenenza di 0.

L'operazione di intersezione tra insiemi fuzzy é definita come il minimo dei gradi di appartenenza, mentre l'operazione di unione é definita come il massimo dei gradi di appartenenza.

2.4.2 Fuzzy systems

I normali insiemi nella logica booleana vengono trasformati in insiemi fuzzy, che consistono nel suddividere l'universo considerato in sottoinsieme definiti da funzioni di appartenenza. La membership function assegna a ogni elemento un grado di appartenenza a ciascun sottoinsieme, che rappresenta la sua "appartenenza" a quel sottoinsieme. Questo grado ha un valore compreso tra 0 e 1.

Un elemento può avere più di una membership function, e quindi appartenere a più sottoinsiemi con gradi di appartenenza diversi.

Algoritmo di inferenza fuzzy

- Definire le variabili linguistiche e i loro insiemi fuzzy associati (initialization)
- Costruire le membership function per ciascun insieme fuzzy (initialization)
- Costruire le regole basi (initialization): delle regole linguistiche if then che collegano le variabili di input alle variabili di output, utilizzando operatori logici fuzzy (come AND, OR, NOT) per combinare le condizioni.
- Convertire i valori di input in gradi di appartenenza utilizzando le membership function (fuzzification)
- Valutazione delle regole secondo le regole base (inference)
- Combinare i risultati per ciascuna regola (inference)
- Convertire l'output fuzzy in un valore crisp utilizzando un metodo di defuzzificazione (defuzzification)

Fuzzy rules Sample fuzzy rules for air conditioning:

IF (temperature is cold OR too-cold) AND (target is warm) THEN command is heat.

IF (temperature is hot OR too-hot) AND (target is warm) THEN command is cool.

IF (temperature is warm) AND (target is warm) THEN command is no-change.

Operazioni Fuzzy OR: MAX:

$$\mu_{A \cup B}(x) = \max(\mu_A(x), \mu_B(x))$$

ASUM:

$$\mu_{A \cup B}(x) = \mu_A(x) + \mu_B(x) - \mu_A(x) \cdot \mu_B(x)$$

BSUM:

$$\mu_{A \cup B}(x) = \min(1, \mu_A(x) + \mu_B(x))$$

AND: MIN:

$$\mu_{A \cap B}(x) = \min(\mu_A(x), \mu_B(x))$$

PROD:

$$\mu_{A \cap B}(x) = \mu_A(x) \cdot \mu_B(x)$$

BDIF:

$$\mu_{A \cap B}(x) = \max(0, \mu_A(x) + \mu_B(x) - 1)$$

Accumulation methods Maximum:

$$\mu_{\text{acc}}(x) = \max \mu_A(x), \mu_B(x)$$

Bounded sum:

$$\mu_{\text{acc}}(x) = \min(1, \mu_A(x) + \mu_B(x))$$

Normalized sum:

$$\mu_{\text{acc}}(x) = \frac{\mu_A(x) + \mu_B(x)}{\max 1, \max \{\mu_A(x'), \mu_B(x')\}}$$

Esempio robot che segue una linea

Chapter 3

Reti Neurali

3.1 Neural Network Concepts and Paradigms

Le reti neurali sono sistemi artificiali di modelli che derivano dallo studio del sistema nervoso dei vertebrati, in particolare del cervello umano. Si usa nell'ambiente dividere la storia delle reti neurali in 3 paradigmi principali:

- Age of Camelot
- Dark Age
- The Renaissance

3.1.1 Age of Camelot

Il primo paradigma, chiamato "Age of Camelot", si riferisce al periodo iniziale di sviluppo delle reti neurali, che va dagli anni '40 agli anni '60. Durante questo periodo, i ricercatori hanno iniziato a esplorare l'idea di creare modelli artificiali ispirati al funzionamento del cervello umano, ma le capacità computazionali e la comprensione del cervello erano ancora limitate, quindi i progressi erano lenti e le applicazioni pratiche erano limitate.

Come il dato (stimolo elettrico) viene elaborato da un neurone biologico, così il dato (input) viene elaborato da un neurone artificiale. Il neurone artificiale riceve un input, lo elabora attraverso una funzione di attivazione e produce un output.

Nel neurone biologico ci sono il corpo cellulare (soma) che sarebbero i nodi, dendriti che sarebbero gli input, e le sinapsi che sarebbero i pesi, e l'assone che sarebbe l'output.

Tutti i segnali input passano attraverso i pesi (che rappresentano le sinapsi) e vengono sommati, e poi passano attraverso una funzione di attivazione che

determina se il neurone si attiva o meno, producendo un output.

Sigmoid é una funzione di attivazione che mappa qualsiasi input in un intervallo compreso tra 0 e 1, ReLU é una funzione di attivazione che restituisce 0 per input negativi e restituisce l'input stesso per input positivi, e Tanh é una funzione di attivazione che mappa qualsiasi input in un intervallo compreso tra -1 e 1.

Peso sinapsico é un valore numerico che rappresenta la forza o l'importanza di una connessione tra due neuroni, e viene utilizzato per modulare il segnale che passa attraverso quella connessione. Quindi si aggiornano.

Il postulato di Hebb (Hebbian Learning) é una teoria che suggerisce che le connessioni sinaptiche tra i neuroni si rafforzano quando i neuroni coinvolti sono attivi contemporaneamente, e si indeboliscono quando non lo sono. Questo principio è spesso riassunto con la frase "neuroni che si attivano insieme, si connettono insieme" (in inglese "neurons that fire together, wire together").

Long Term Potentiation (LTP) é un processo biologico che si verifica nelle sinapsi del cervello, in cui la forza della connessione tra due neuroni aumenta in modo duraturo dopo che i neuroni sono stati attivati insieme per un certo periodo di tempo.

Per la rete neurale artificiale dobbiamo innanzitutto definire una architettura, un modo di tradurre un dato capibile dal computer (quindi numerico, codici, sequenze) quella che sarà una fotografia.

Regole di apprendimento per aggiornare i pesi, e una funzione di attivazione che ci dice se un neurone si attiva o no.

Nel 1943 Warren McCulloch e Walter Pitts pubblicarono un articolo intitolato "A Logical Calculus of the Ideas Immanent in Nervous Activity", in cui proposero un modello matematico di neurone artificiale, noto come "neurone di McCulloch-Pitts". Questo modello rappresentava un neurone come una unità di elaborazione che riceveva input binari (0 o 1) e produceva un output binario (0 o 1) in base a una funzione di attivazione lineare.

Nel 1957 Frank Rosenblatt sviluppò il "Perceptron", un modello di rete neurale artificiale che poteva essere addestrato a riconoscere pattern semplici. Il Perceptron era in grado di apprendere a classificare dati lineari, ma aveva limitazioni significative, come l'incapacità di risolvere problemi non lineari, come il famoso problema XOR.

Il Perceptrone fa parte delle supervised ANN (Artificial Neural Networks), ovvero reti neurali che vengono addestrate con dati etichettati, in cui ogni input è associato a un output desiderato.

Gli ingressi possono essere 0 o 1 (come nel caso del Perceptron) o possono essere valori continui (come nelle reti neurali più moderne).

Il perceptrone usato come logica booleana é un esempio di rete neurale artificiale che può essere utilizzata per implementare funzioni logiche, come AND, OR e NOT, ma non è in grado di implementare funzioni logiche più complesse, come XOR, a causa della sua limitazione nella separazione lineare dei dati.

Pseudocodifce per Perceptron learning

```
initialize weights w and bias b to small random values
for each training example (x, y):
    compute the output: output = activation_function(w * x + b)
    update the weights and bias:
        w = w + learning_rate * (y - output) * x
        b = b + learning_rate * (y - output)
```

Nel libro "Perceptrons" del 1969, Marvin Minsky e Seymour Papert analizzarono le limitazioni del Perceptrone, sottolineando in particolare la sua incapacità di risolvere il problema XOR. Questo lavoro ebbe un impatto significativo sulla comunità scientifica, portando a una diminuzione dell'interesse per le reti neurali e contribuendo alla fase successiva, nota come "Dark Age". E' single layer perché ha un solo strato di neuroni, e feedforward perché i dati fluiscono in una sola direzione, dagli input agli output, senza cicli o feedback.

L'obiettivo del Perceptrone era quello di trovare un iperpiano che separasse i dati in due classi, ma non era in grado di risolvere problemi non lineari, come il problema XOR, che richiede la separazione di dati che non possono essere divisi da un iperpiano lineare.

Il problema xor é un problema di classificazione in cui i dati sono organizzati in modo tale che non possono essere separati da una linea retta (o iperpiano) in uno spazio bidimensionale.

L'idea per risolvere il problema XOR è stata quella di introdurre strati nascosti (hidden layers) nella rete neurale, (il -1 proviene dal bias, pesi sinapsici). Questi perceptroni funzionano come elementi che sono in grado di gestire una risposta soddisfacente solo se hanno un allenamento dato da un buon numero di dati, e se hanno un numero di neuroni nascosti sufficienti a rappresentare la complessità del problema.

Prima i dati non erano sufficienti per offrire un allenamento adeguato e poi le capacità computazionali erano limitate, quindi non era possibile costruire reti neurali con un numero sufficiente di strati nascosti e neuroni per risolvere problemi complessi come XOR.

La tecnologia ha messo a disposizione ai ricercatori bacini di dati sempre più grandi e capacità computazionali sempre più potenti (visto che ho più dati da calcolare), permettendo lo sviluppo di reti neurali profonde (deep learning) con

molti strati nascosti, che sono in grado di risolvere problemi molto complessi e di apprendere rappresentazioni gerarchiche dei dati.

Quando la regione è convessa significa che è possibile trovare un punto di minimo globale, mentre quando è non convessa ci possono essere molti minimi locali, e quindi è più difficile trovare il minimo globale.

Ogni ingresso è collegato a ogni neurone dello strato nascosto, e ogni neurone dello strato nascosto è collegato a ogni neurone dello strato di output.

Il ruolo della funzione di attivazione è quello di introdurre non linearità nel modello, permettendo alla rete neurale di apprendere e rappresentare relazioni complesse tra i dati.

Possibili funzioni di attivazione:

- Unit Step Function: restituisce 1 se l'input è maggiore di una soglia, altrimenti restituisce 0.
- Sigmoid Function: restituisce un valore compreso tra 0 e 1
- Linear Function: restituisce l'input stesso, senza modifiche
- ReLU (Rectified Linear Unit): restituisce 0 per input negativi e poi è lineare per input positivi
- Tanh (Hyperbolic Tangent): restituisce un valore compreso tra -1 e 1

Softmax in matematica è una funzione di attivazione che trasforma un vettore di valori reali in un vettore di probabilità, dove ogni elemento del vettore di output rappresenta la probabilità che l'input appartenga a una determinata classe.

La funzione softmax è spesso utilizzata nell'ultimo strato di una rete neurale per problemi di classificazione multi-classe, perché consente di interpretare l'output della rete come una distribuzione di probabilità sulle classi di output.

è una generalizzazione della funzione logistica che per un vettore a k-dimensioni di valori reali, ...

ReLU è una funzione di attivazione che è solamente $R(x) = \max(0, x)$ if $x \geq 0$
 $R(x) = x$ if $x < 0$, quindi restituisce 0 per input negativi e restituisce l'input stesso per input positivi.

Il multi-layer perceptron (MLP) è una rete neurale artificiale composta da più strati di neuroni, abbiamo i neuroni di ingresso, gli hidden layer e i neuroni di output.

In generale non ci sono connessioni tra neuroni dello stesso strato, e i dati fluiscono in una sola direzione, dagli input agli output, senza cicli o feedback.

Back Propagation: Learning Problems: Un possibile learning algorithm potrebbe essere :

1. Inizializzare i pesi sinaptici con valori casuali
2. Presentare un pattern e l'output della rete
3. Calcolare l'output
4. Aggiornare i pesi sinaptici in base alla differenza tra l'output calcolato e l'output desiderato

Tra l'output calcolato e l'output desiderato c'è una differenza, che è chiamata "errore". Il problema è che non sappiamo come aggiornare i pesi sinaptici in modo efficace per ridurre l'errore, soprattutto quando abbiamo più strati nascosti.

La regola adoperata si chiama Delta Rule, che è una regola di apprendimento supervisionato che viene utilizzata per addestrare reti neurali a strato singolo, come il Perceptron.

In ingresso il segnale elabora l'output, e in uscita c'è un errore, che è la differenza tra l'output desiderato e l'output calcolato.

La regola Delta aggiorna i pesi sinaptici in modo da ridurre l'errore, utilizzando la formula:

$$\Delta w = \alpha(t_i - y_i)g'(h_i)x_i$$

dove Δw è la variazione del peso sinaptico, α è il tasso di apprendimento, t_i è l'output desiderato, y_i è l'output calcolato e x_i è l'input.

Questa regola è comunemente semplificata come:

$$\Delta w = \alpha(t_i - y_i)x_i$$

La regola Delta è efficace per addestrare reti neurali a strato singolo, ma non è adatta per reti neurali a più strati, poiché non fornisce un modo per calcolare l'errore nei strati nascosti.

Il gradiente vettorializza uno scalare, in questo caso ho vari pesi sinapsici e vettorializzo l'errore scalare, e questo mi permette di aggiornare tutti i pesi sinaptici in modo efficiente.

$$\Delta w = -\eta \frac{\delta E}{\delta w}$$

dove Δw è la variazione del peso sinaptico, η è il tasso di apprendimento, E è la funzione di errore e $\frac{\delta E}{\delta w}$ è il gradiente dell'errore rispetto al peso sinaptico.

Note on Learning Rate

3.1.2 Dark Age

Il secondo paradigma, chiamato "Dark Age", si riferisce al periodo dagli anni '70 agli anni '80, durante il quale l'interesse per le reti neurali è diminuito notevolmente. Questo è stato causato da una serie di fattori, tra cui la mancanza di

risultati significativi, le limitazioni computazionali e la concorrenza di altre tecniche di intelligenza artificiale, come i sistemi esperti basati su regole. Durante questo periodo, le reti neurali sono state spesso considerate un campo di ricerca marginale e hanno ricevuto poca attenzione da parte della comunità scientifica.

3.1.3 The Renaissance

Il terzo paradigma, chiamato "The Renaissance", si riferisce al periodo dagli anni '90 ad oggi, durante il quale le reti neurali hanno conosciuto una rinascita significativa. Questo è stato reso possibile da una serie di fattori, tra cui l'aumento delle capacità computazionali, la disponibilità di grandi quantità di dati e lo sviluppo di nuove architetture di reti neurali, come le reti neurali profonde (deep learning). Durante questo periodo, le reti neurali sono diventate una delle tecniche più popolari e di successo nell'ambito dell'intelligenza artificiale, con applicazioni in una vasta gamma di settori, tra cui il riconoscimento vocale, la visione artificiale, la traduzione automatica e molti altri.

The back-propagation algorithm è stato introdotto negli anni '80 da David Rumelhart, Geoffrey Hinton e Ronald Williams, ed è diventato uno degli algoritmi di apprendimento più importanti per le reti neurali a più strati. Primo passo è quello di calcolare l'output di uscita, e il secondo passo usare la delta rule per aggiornare i pesi sinaptici. Questo processo viene ripetuto per ogni pattern di addestramento, e i pesi sinaptici vengono aggiornati in modo iterativo fino a quando la rete neurale non raggiunge un livello di prestazioni soddisfacente.

Momentum Descent è una variante del metodo di discesa del gradiente che introduce un termine di "momentum" per accelerare la convergenza e ridurre il rischio di rimanere bloccati in minimi locali.

Quando una rete neurale deve gestire degli input totalmente differenti in una direzione, corriamo il rischio di avere differenze nette dalle grandezze che devo gestire, e questo può portare a problemi di convergenza durante l'addestramento. Quello che si fa è una normalizzazione, in modo che tutte le grandezze sono gestite in modo conveniente, e questo aiuta a migliorare la convergenza durante l'addestramento della rete neurale.

Il gradient descending è una procedura iterativa utilizzata per ottimizzare i pesi sinaptici di una rete neurale, al fine di minimizzare la funzione di errore. L'algoritmo di discesa del gradiente funziona calcolando il gradiente della funzione di errore rispetto ai pesi sinaptici, e poi aggiornando i pesi sinaptici in direzione opposta al gradiente, con un passo di aggiornamento determinato dal tasso di apprendimento.

Come si svolge la fase del training: Abbiamo un bacino di dati che abbiamo pulito dal rumore, dobbiamo dimensionare il training, ogni volta che faccio passare tutti i dati di addestramento e ottengo tutte le uscite ho completato un'epoca. Uso un approccio Batch, quindi faccio passare tutti i dati di addestramento prima di aggiornare i pesi sinaptici, e questo mi permette di avere una stima più accurata del gradiente.

I batch sono partizionamenti di tutto il training. On-line o Batch Training? On-line training é un approccio in cui i pesi sinaptici vengono aggiornati dopo ogni singolo esempio di addestramento, mentre batch training é un approccio in cui i pesi sinaptici vengono aggiornati dopo aver elaborato un intero batch di esempi di addestramento.

L'on-line training può essere più veloce e adattivo, ma può essere più rumoroso e meno stabile, mentre il batch training può essere più stabile e fornire una stima più accurata del gradiente, ma può essere più lento e richiedere più memoria.

Overfitting: é un fenomeno che si verifica quando una rete neurale impara a memoria i dati di addestramento, perdendo la capacità di generalizzare a nuovi dati.

Pruning: é una tecnica utilizzata per ridurre la complessità di una rete neurale, rimuovendo i pesi sinaptici che non contribuiscono in modo significativo alla prestazione della rete.

Regolarizzazione dei pesi: é una tecnica utilizzata per prevenire l'overfitting, aggiungendo un termine di penalizzazione alla funzione di errore che incoraggia i pesi sinaptici a essere piccoli.

Jittering: allenare la rete non con dati puri ma con dati con rumore controllato (rumore bianco).

Quando si campiona il segnale si potrebbe introdurre un rumore periodico, e può succedere quando c'è un rumore di fondo gne gne.

3.2 Reti Neurali Non Supervisionati

Tra tutte le reti delle reti non supervisionate, ci sta la Self Organizing Maps (SOMs), che è una rete neurale artificiale che utilizza un algoritmo di apprendimento non supervisionato per mappare dati ad alta dimensione in uno spazio a bassa dimensione, preservando le relazioni topologiche tra i dati.

Questa rete funziona con un apprendimento chiamato "competitive learning". Vengono usati quando ci sono molti dati disorganizzati, e si vuole trovare una rappresentazione più compatta e organizzata dei dati, come ad esempio per la visualizzazione o la riduzione della dimensionalità.

Si prestano alla compressione dei dati tramite la quantizzazione vettoriale.

3.2.1 Architettura Schematica di KNN SOM

Quando vogliamo stimolare un uscita di un neurone j -esimo dobbiamo avere uno stimolo iniziale che è dato da un input, e poi dobbiamo avere una connessione che è data da un peso sinaptico, e poi dobbiamo avere una funzione di attivazione che ci dice se il neurone si attiva o no.

y_j come quantità di stimolo che arriva al neurone j -esimo, w_{ij} come peso sinaptico che collega il neurone i -esimo all'uscita del neurone j -esimo, e x_i come input che arriva al neurone i -esimo.

Il valore dei pesi fissi viene aggiornato durante la fase di addestramento, in cui i pesi vengono modificati in base alla distanza tra l'input e il neurone vincente (il neurone con il peso più vicino all'input).

Il neurone vincente viene identificato calcolando la distanza tra l'input e i pesi di ciascun neurone nella mappa, e selezionando il neurone con la distanza minima. Una volta identificato il neurone vincente, i pesi di quel neurone e dei suoi vicini vengono aggiornati per avvicinarli all'input, utilizzando una formula di aggiornamento che dipende dalla distanza tra il neurone e l'input, e da un tasso di apprendimento che diminuisce nel tempo.

I neuroni vicini al vicinato hanno pesi positivi, mentre i neuroni lontani hanno pesi negativi, e questo aiuta a preservare le relazioni topologiche tra i dati.

Se io prendo un neurone centrale, definisco il vicinato con il laplaciano gaussiano. Il neurone con più alto output viene chiamato neurone vincente, e i pesi di quel neurone e dei suoi vicini vengono aggiornati per avvicinarli all'input, utilizzando una formula di aggiornamento che dipende dalla distanza tra il neurone e l'input, e da un tasso di apprendimento che diminuisce nel tempo.

Un isomorfismo è una mappa che preserva le relazioni topologiche tra i dati, quindi se due punti sono vicini nello spazio ad alta dimensione, saranno vicini anche nello spazio a bassa dimensione.

I neuroni di uscita non sono connessi tra loro, quindi non c'è competizione tra i neuroni di uscita, ma piuttosto una competizione tra i neuroni di ingresso per attivare un neurone di uscita.

Si possono usare due tecniche per identificare il neurone vincente:

- Il neurone con il massimo output, che è il neurone con il peso più vicino all'input
- Si sceglie il neurone il cui peso vettore è più simile al vettore di input, utilizzando una misura di distanza come la distanza euclidea o la distanza di Manhattan

Metodo 1

$$y_i = \sum w_{ij}x_j$$

Dove y_i è l'output del neurone i-esimo, w_{ij} è il peso sinaptico che collega il neurone j-esimo all'uscita del neurone i-esimo, e x_j è l'input che arriva al neurone j-esimo.

Posso scriverlo anche come:

$$y_i = W_j \cdot X = |W_j||X| \cos \theta$$

Dove W_j è il vettore dei pesi del neurone j-esimo, X è il vettore di input, e θ è l'angolo tra i due vettori.

Metodo 2 Il neurone vincente su input X è quello la quale ha il peso vettore simile a X .

$$j_0 : DIS(W_{j_0}, X) < DIS(W_j, X) \forall j \neq j_0$$

Ci sono diversi modi per calcolare la distanza (DIS):

- Euclidean Distance
- Road Distance (Manhattan Distance)
- Hamming Distance (only for binary vectors)

Learning Rule

$$\Delta W(t) = \alpha(X - W)$$

Dove $\Delta W(t)$ è la variazione del peso sinaptico al tempo t , α è il tasso di apprendimento, X è il vettore di input e W è il vettore dei pesi del neurone vincente.

Il tasso di apprendimento α è un parametro che controlla la velocità con cui i pesi vengono aggiornati durante l'addestramento, e di solito diminuisce nel tempo per consentire alla rete di convergere verso una soluzione stabile.

Il processo di addestramento continua fino a quando i pesi non convergono a valori stabili, o fino a quando non viene raggiunto un numero massimo di iterazioni.

Neighborhood update Il raggio di interazione è l'insieme dei neuroni che hanno una distanza più piccola di un certo valore (raggio) dal neurone vincente, e i pesi di questi neuroni vengono aggiornati in modo simile a quelli del neurone vincente, ma con un tasso di apprendimento che diminuisce con la distanza dal neurone vincente.?????

3.2.2 Kohonen Learning Algorithm

.....

3.3 RNN - Recurrent Neural Networks

Creare una situazione in cui il neurone viene eccitato anche attraverso un feedback loop che la sua stessa uscita produce. Quindi in ingresso ha i dati che arrivano da altri neuroni e anche i dati che arrivano da lui stesso.

Abbiamo dei feedback che possono provenire (oltre a loro stessi) anche da neuroni dello stesso strato, qui il flusso informativo è più articolato. Fully connected è quando ogni neurone è connesso a ogni altro neurone, quindi anche a se stesso.

Abbiamo un input all'istante t e uno stato nascosto che rappresenta la memoria della rete, e un output che dipende sia dall'input che dallo stato nascosto. Queste reti non hanno una memoria a lungo termine, quindi si è pensato di creare una memoria più a lungo termine attraverso le reti LSTM (Long Short-Term Memory).

3.3.1 LSTM - Long Short-Term Memory

Le RNN usano le informazioni passate per migliorare le prestazioni di una rete neurale su input attuali e futuri, queste reti contengono uno stato nascosto che consentono alla rete di mantenere una memoria a breve termine, ma non sono in grado di gestire efficacemente la memoria a lungo termine a causa del problema del gradiente che scompare.

Si aggiungono per le LSTM delle porte aggiuntive che consentono alla rete di mantenere e aggiornare una memoria a lungo termine, permettendo alla rete di apprendere e ricordare informazioni per periodi di tempo più lunghi. Due input c_{t-1} e h_{t-1} , e due output h_t e c_t . Le porte (gate) f , g , i , o sono funzioni di attivazione che controllano il flusso di informazioni all'interno della cella LSTM.

La porta di dimenticanza (forget gate) f decide quali informazioni devono essere dimenticate dalla memoria a lungo termine, la porta g (memory cell), la porta di input (input gate) i decide quali nuove informazioni devono essere aggiunte alla memoria a lungo termine, la porta di output (output gate) o decide quali informazioni devono essere restituite come output.

Per la musica si usano le RNN, perché la musica è una sequenza di note che si sviluppa nel tempo, e le RNN sono in grado di catturare le dipendenze temporali tra le note, per esempio, se una nota è seguita da un'altra nota, la RNN può imparare a prevedere la seconda nota in base alla prima nota. Però non si ricordavano le note di istanti precedenti quindi si è pensato di usare le

LSTM, che hanno una memoria a lungo termine, e quindi possono ricordare le note di istanti precedenti e prevedere le note future in modo più accurato.

3.4 Transformer

Sia LSTM che RNN adoperano un processamento sequenziale, viceversa i Transformers introducendo l'idea del multihead sono in grado di addestrarsi in parallelo, e questo è un vantaggio significativo in termini di efficienza computazionale, soprattutto quando si lavora con grandi quantità di dati.

"Attention is all you need" è il titolo di un articolo del 2017 che ha introdotto il concetto di Transformer, un'architettura di rete neurale che si basa interamente sul meccanismo di auto-attenzione, senza l'uso di convoluzioni o ricorrenze.

Il punto di forza dell'architettura Transformer risiede nell'utilizzo di meccanismi di attenzione, in particolare quello dell'autoattenzione. Sfrutta l'autoattenzione per modellare le dipendenze tra i token in una sequenza, indipendentemente dalla loro distanza reciproca. Questo consente al modello di catturare relazioni a lungo raggio tra i token, migliorando la capacità di comprendere il contesto e le relazioni semantiche all'interno della sequenza.

Riescono a operare in modo parallelo, quindi l'addestramento sfrutta l'informazione che viene operata simultaneamente su più token, invece nelle RNN e LSTM l'addestramento è sequenziale, quindi ogni token viene elaborato uno alla volta, e questo può essere un collo di bottiglia in termini di efficienza computazionale, soprattutto quando si lavora con grandi quantità di dati.

I Transformer sono in grado di collegare le informazioni lontano nel tempo o testo, invece nelle RNN e LSTM (che ha uno svuotatore di memoria) non è possibile. Nonostante ciò le RNN e LSTM sono ancora utilizzate in alcune applicazioni (a volte per le risorse limitate), come il riconoscimento vocale e la generazione di testo, dove la capacità di catturare le dipendenze temporali è importante, ma i Transformer stanno diventando sempre più popolari in molte applicazioni di elaborazione del linguaggio naturale e di visione artificiale, grazie alla loro capacità di catturare relazioni a lungo raggio e alla loro efficienza computazionale.

Concetto dell'auto-attenzione (Transformer): è un meccanismo che consente a una rete neurale di concentrarsi su parti specifiche dell'input quando elabora i dati, invece di elaborare l'intero input in modo uniforme. Questo è particolarmente utile per compiti come la traduzione automatica, dove alcune parole o frasi possono essere più importanti di altre per comprendere il significato del testo.

Il meccanismo di auto-attenzione funziona calcolando un peso di attenzione per ogni elemento dell'input, che indica quanto quell'elemento è rilevante per il compito in questione. Questi pesi di attenzione vengono poi utilizzati per combinare gli elementi dell'input in modo da produrre un output che tiene conto delle parti più importanti dell'input.

3.4.1 Self Attention

Tradizionalmente, per rappresentare le parole come vettori si utilizzavano metodi come One-Hot Encoding, Bag of Words e TF-IDF, che rappresentano le parole come vettori sparsi e non catturano le relazioni semantiche tra le parole. Si basavano principalmente sulla frequenza e posizione delle parole nei documenti, senza considerare il contesto in cui le parole appaiono.

Per superare queste limitazioni, la comunità NLP ha adottato gli embedding di parole, che rappresentano le parole come vettori densi in uno spazio continuo, catturando le relazioni semantiche tra le parole. Per esempio la parola *re* è più vicina alla parola *regina* che alla parola *mela*. Ma ancora una parola poteva avere più significati, esempio *light* significa sia *leggero* che *luce*.

Qui entra l'autoattenzione, che consente al modello di concentrarsi su parti specifiche dell'input quando elabora i dati, e quindi di catturare il contesto in cui le parole appaiono. Considera l'intero contesto della sequenza di input.

Il blocco dell'autoattenzione opera su tre componenti (vettori) fondamentali:

- Query (Q): rappresenta la parola o il token per cui stiamo calcolando l'attenzione. Per ogni parola della sequenza, il modello genera un vettore di interrogazione che viene poi utilizzato per confrontare quella parola con le altre parole della sequenza e calcolare i pesi di attenzione.
- Key (K): rappresenta le parole o i token che stiamo confrontando con la query per calcolare l'attenzione.
- Value (V): rappresenta le parole o i token che stiamo combinando per produrre l'output dell'attenzione.

Quindi attraverso dei pesi di query e key si calcolano i pesi di attenzione, che indicano quanto ogni parola della sequenza è rilevante per la parola di interesse (query), e poi questi pesi di attenzione vengono utilizzati per combinare i valori (value) delle parole della sequenza, in modo da produrre un output che tiene conto delle parti più importanti dell'input.

Passaggi per il blocco Autoattenzione

- Calcolare i vettori di query, key e value per ogni parola della sequenza di input, utilizzando matrici di peso apprese durante l'addestramento.
- Calcolare i punteggi di attenzione per ogni parola della sequenza, confrontando i vettori di query con i vettori di key, e normalizzando i risultati con una funzione softmax.
- Scalatura dei punteggi di attenzione per evitare problemi di stabilità numerica, dividendo i punteggi per la radice quadrata della dimensione dei vettori di key.
- Applicare la funzione softmax ai punteggi di attenzione scalati per ottenere i pesi di attenzione, che indicano quanto ogni parola della sequenza è rilevante per la parola di interesse (query).
- Utilizzare i pesi di attenzione per combinare i vettori di value delle parole della sequenza, in modo da produrre un output che tiene conto delle parti più importanti dell'input.

3.4.2 Multi-Head Attention

Il meccanismo di multi-head attention è un'estensione del meccanismo di auto-attenzione che consente al modello di catturare diverse relazioni tra le parole in una sequenza, utilizzando più "teste" di attenzione.

Invece di calcolare un singolo set di pesi di attenzione, il modello calcola più set di pesi di attenzione in parallelo, ognuno con le proprie matrici di peso per query, key e value.

Questo consente al modello di catturare diverse relazioni tra le parole in una sequenza, come relazioni sintattiche, semantiche e di dipendenza, migliorando la capacità del modello di comprendere il contesto e le relazioni semantiche all'interno della sequenza.

Il meccanismo di multi-head attention funziona calcolando più set di pesi di attenzione in parallelo, e poi combinando i risultati di ciascuna testa di attenzione per produrre l'output finale.

Ogni testa di attenzione può concentrarsi su aspetti diversi della sequenza di input, e combinando i risultati di più teste di attenzione, il modello può catturare una gamma più ampia di relazioni tra le parole in una sequenza, migliorando la capacità del modello di comprendere il contesto e le relazioni semantiche all'interno della sequenza.

Le fasi sono:

- Proiezione lineare: per ogni testa di attenzione, proiettare i vettori di query, key e value in spazi di dimensione inferiore utilizzando matrici di peso apprese durante l'addestramento.
- Calcolo dell'attenzione: per ogni testa di attenzione, calcolare i pesi di attenzione utilizzando i vettori di query, key e value proiettati, seguendo i passaggi descritti nel blocco di auto-attenzione.

- Concatenazione: concatenare i risultati di tutte le teste di attenzione per ottenere un singolo vettore di attenzione combinato.
- Proiezione finale: proiettare il vettore di attenzione combinato in uno spazio di dimensione originale utilizzando una matrice di peso appresa durante l'addestramento, per ottenere l'output finale del meccanismo di multi-head attention.

Questo tipo di attenzione è importante perché consente al modello di catturare diverse relazioni tra le parole in una sequenza, migliorando la capacità del modello di comprendere il contesto e le relazioni semantiche all'interno della sequenza, e quindi migliorando le prestazioni del modello su compiti di elaborazione del linguaggio naturale e di visione artificiale.

3.4.3 Masked Multi-head Attention

Il meccanismo di masked multi-head attention è una variante del meccanismo di multi-head attention che viene utilizzata nei modelli di generazione di testo, come i modelli di linguaggio autoregressivi.

In un modello di linguaggio autoregressivo, il modello genera testo un token alla volta, e quindi non può utilizzare informazioni future per generare il token corrente. Il meccanismo di masked multi-head attention risolve questo problema mascherando i token futuri durante il calcolo dei pesi di attenzione, in modo che il modello possa concentrarsi solo sui token passati e presenti.

Il meccanismo di masked multi-head attention funziona calcolando i pesi di attenzione come nel meccanismo di multi-head attention, ma con l'aggiunta di una maschera che impedisce al modello di accedere ai token futuri durante il calcolo dei pesi di attenzione.

La maschera viene applicata ai punteggi di attenzione prima della normalizzazione softmax, impostando i punteggi di attenzione per i token futuri a un valore molto basso (ad esempio, $-\infty$) in modo che i pesi di attenzione per quei token siano praticamente zero dopo l'applicazione della funzione softmax.

Questo consente al modello di concentrarsi solo sui token passati e presenti durante la generazione del testo, migliorando la coerenza e la qualità del testo generato.

Processo di generazione del testo con masked multi-head attention:

- Mascheramento dei token futuri: durante il calcolo dei pesi di attenzione, applicare una maschera che impedisce al modello di accedere ai token futuri, impostando i punteggi di attenzione per i token futuri a un valore molto basso (ad esempio, $-\infty$).
- Implementazione della maschera: durante il calcolo dei punteggi di attenzione, aggiungere la maschera ai punteggi di attenzione prima della normalizzazione softmax, in modo che i pesi di attenzione per i token futuri siano praticamente zero dopo l'applicazione della funzione softmax.

- Applicazione su più teste: applicare la maschera a tutte le teste di attenzione in parallelo, assicurando che il modello non possa accedere ai token futuri durante il calcolo dei pesi di attenzione per tutte le teste.

3.4.4 Transformer e musica

I Transformer sono stati utilizzati con successo anche nella generazione di musica, grazie alla loro capacità di catturare relazioni a lungo raggio e di comprendere il contesto all'interno di una sequenza.

Per applicare i Transformer alla generazione di musica, è necessario rappresentare la musica in un formato che possa essere elaborato dal modello. Una comune rappresentazione della musica è il formato MIDI, che rappresenta la musica come una sequenza di eventi, come note, durate e dinamiche.

Una volta che la musica è stata rappresentata in un formato adatto, è possibile addestrare un modello di Transformer sulla sequenza di eventi musicali, utilizzando tecniche di addestramento simili a quelle utilizzate per la generazione di testo.

Il modello di Transformer impara a prevedere il token musicale successivo in base ai token musicali precedenti, e può essere utilizzato per generare nuove sequenze musicali che seguono lo stile e la struttura della musica di addestramento.

I Transformer hanno dimostrato di essere efficaci nella generazione di musica, producendo risultati di alta qualità che possono essere indistinguibili dalla musica composta da esseri umani, e stanno diventando sempre più popolari come strumento per la composizione musicale assistita da intelligenza artificiale.

Chapter 4

Reti Generative

4.1 Reti Generative Avversarie (GAN)

Il loro approccio é del 2014 e sono capaci di generare contenuti estremamente realistici come volti umani inesistenti, opere d'arte e video (i cosiddetti "deep-fakes").

Posso avere impatti artistici con IA e avere tutti i problemi di copyright e etici o usarli come integrazione artistica.

Il meccanismo é realizzato attraverso due giocatori: un **generatore** (Falsario) e un **discriminatore** (Esperto). Il generatore crea dati falsi, mentre il discriminatore cerca di distinguere tra dati reali e falsi. Entrambi i modelli vengono addestrati simultaneamente, con il generatore che cerca di ingannare il discriminatore e quest'ultimo che cerca di migliorare la sua capacità di identificare i dati falsi.

Una rete (Generatore) produce in uscita dei dati (riferiti a un contesto specifico, ad esempio immagini (quindi matrice pixel) o testo) e questi dati vengono dati in input a un'altra rete (discriminatore) che deve capire se sono dati reali o dati generati.

Il generatore riceve in ingresso le conferme o le smentite del discriminatore e cerca di migliorare la sua capacità di ingannare il discriminatore.

Inganno davvero il discriminatore, quando il discriminatore non riesce più a distinguere tra dati reali e dati generati (quindi da 50% vero e 50% falso), allora il generatore ha raggiunto il suo obiettivo.

Questo processo é chiamato "gioco a somma zero" perché il guadagno di un giocatore è esattamente bilanciato dalla perdita dell'altro.

Matematicamente, questo processo é descritto come un problema di **minimax**, dove l'obiettivo é trovare un punto di equilibrio in cui nessuno dei due giocatori può migliorare la propria posizione senza peggiorare quella dell'altro.

Minimax Il minimax é una strategia usata soprattutto nei giochi a informazione completa e a somma zero (dove se io vinco (+1) tu perdi (-1) e viceversa).

Il concetto è quello di scegliere la mossa che minimizza la massima perdita possibile.

In pratica é come giocare a scacchi. Tu non scegli la mossa che ti dà il massimo guadagno, ma quella che ti dà il minimo danno, considerando che l'avversario farà la mossa migliore per lui.

Nelle GAN il generatore cerca di minimizzare la probabilità che il discriminatore indovini che i dati sono falsi, mentre il discriminatore cerca di massimizzare questa probabilità.

Equilibrio di Nash Il concetto di equilibrio di Nash si riferisce a una situazione in cui nessun giocatore può migliorare la propria posizione cambiando unilateralmente la propria strategia, considerando le strategie degli altri giocatori.

É una situazione di "stallo perfetto". Se io so cosa farai tu, e tu sai cosa farò io, allora nessuno dei due ha un incentivo a cambiare la propria strategia.

Un esempio é il dilemma del prigioniero, dove due prigionieri devono decidere se collaborare o tradire l'altro. Se entrambi collaborano, ottengono una pena ridotta. Se uno tradisce e l'altro collabora, il traditore ottiene una pena ancora più ridotta, mentre il collaboratore ottiene una pena più severa. Se entrambi tradiscono, ottengono una pena moderata. L'equilibrio di Nash in questo caso è che entrambi i prigionieri scelgono di tradire, anche se sarebbe meglio per entrambi se collaborassero.

Quindi mentre il minimax ti dice come giocare per proteggerti, l'equilibrio di nash descrive dove finisce il gioco.

Nelle GAN quando il generatore produce immagini perfette il discriminatore non riesce più a distinguere tra dati reali e dati generati, quindi si raggiunge un equilibrio di Nash (50 VERE 50 FALSI).

Immaginiamo come se fosse una partita a scacchi infinita. Nel pratico la funzione di costo (obiettivo) é condivisa:

- Il generatore cerca di minimizzare il punteggio del discriminatore. Il suo obiettivo é che il discriminatore assegni 1 (vero) ai suoi falsi
- Il discriminatore cerca di massimizzare la sua capacità di distinguere tra il vero dal falso. Il suo punteggio ideale é 1 per le foto vere e 0 per quelle

create dal generatore.

Esempio pratico: creazione immagine gatto

- Il generatore nota che se mette le orecchie a punta, il discriminatore tende a classificare l'immagine come un gatto. Quindi inizia a creare immagini con orecchie a punta.
- Il generatore non sceglie una strategia azzardata (tipo creare gatto blu sperando che il discriminatore non se ne accorga), ma cerca di migliorare gradualmente le sue immagini basandosi sui feedback ricevuti dal discriminatore.
- Il generatore minimizza la sua perdita massima: ovvero, cerca di produrre l'immagine che ha la minor probabilità possibile di essere scartata, assumendo che il discriminatore stia facendo del suo meglio per identificare i falsi.

Supponiamo che dopo migliaia di tentativi:

- Il generatore produce un'immagine di un gatto che è così realistica che il discriminatore non riesce più a distinguere se è reale o falsa.
- In questo punto, il generatore ha raggiunto il suo obiettivo: ha ingannato con successo il discriminatore.
- Il discriminatore, d'altra parte, ha raggiunto un punto in cui non può migliorare ulteriormente la sua capacità di distinguere tra reale e falso, poiché le immagini generate sono indistinguibili da quelle reali. La sua probabilità di successo è del 50%, il che significa che è in equilibrio di Nash con il generatore.

Se il generatore cambia strategia (ad esempio, inizia a creare cani), il discriminatore potrebbe facilmente identificare queste immagini come false, quindi il generatore non ha alcun incentivo a cambiare la sua strategia una volta raggiunto l'equilibrio di Nash.

Se il discriminatore cambia strategia (ad esempio, inizia a concentrarsi su dettagli specifici come il colore degli occhi), il generatore potrebbe adattarsi per migliorare quei dettagli, ma se le immagini sono già indistinguibili, il discriminatore non ha alcun incentivo a cambiare la sua strategia.

4.1.1 Sintesi Visiva

$$\min_G \max_D V(D, G)$$

Dove:

- G è il generatore, che cerca di minimizzare la funzione di costo.

- D è il discriminatore, che cerca di massimizzare la funzione di costo.
- $V(D, G)$ è la funzione di costo condivisa che rappresenta l'obiettivo del gioco a somma zero tra il generatore e il discriminatore.
- $\min_G$ indica che il generatore cerca di minimizzare la funzione di costo, mentre $\max_D$ indica che il discriminatore cerca di massimizzare la stessa funzione.

L'espressione che si usa:

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p_z} [\log(1 - D(G(z)))]$$

Dove:

- p_{data} è la distribuzione dei dati reali.
- p_z è la distribuzione del vettore di rumore z .

Il primo termine $E_{x \sim p_{data}} [\log D(x)]$ rappresenta l'aspettativa del logaritmo della probabilità che il discriminatore assegna ai dati reali. Il discriminatore cerca di massimizzare questo termine, cercando di assegnare una probabilità alta (vicina a 1) ai dati reali.

Il secondo termine $E_{z \sim p_z} [\log(1 - D(G(z)))]$ rappresenta l'aspettativa del logaritmo della probabilità che il discriminatore assegna ai dati generati dal generatore. Il generatore cerca di minimizzare questo termine, cercando di far sì che il discriminatore assegni una probabilità bassa (vicina a 0) ai dati generati, ovvero cercando di ingannare il discriminatore facendogli credere che i dati generati siano reali.

$\sim$ indica che stiamo campionando da una distribuzione, quindi $x \sim p_{data}$ significa che stiamo campionando dati reali dalla distribuzione dei dati, e $z \sim p_z$ significa che stiamo campionando un vettore di rumore dalla distribuzione del rumore.

Cosa succede durante il training? Due fasi di addestramento:

- **Fase A: Il turno del discriminatore ($\max_D$):** In questa fase, il discriminatore viene addestrato a distinguere tra dati reali e dati generati. Viene presentato con un mix di dati reali (campionati da p_{data}) e dati generati (creati dal generatore $G(z)$). Il discriminatore aggiorna i suoi pesi per massimizzare la probabilità di classificare correttamente i dati reali come veri e i dati generati come falsi. Aggiorna i suoi pesi per alzare $D(x)$ per i dati reali e abbassare $D(G(z))$ per i dati generati.
- **Fase B: Il turno del generatore ($\min_G$):** In questa fase, il generatore viene addestrato a creare dati che ingannino il discriminatore. Viene presentato con un vettore di rumore z campionato da p_z , e produce dati

generati $G(z)$. Il generatore aggiorna i suoi pesi per minimizzare la probabilità che il discriminatore classifichi i dati generati come falsi, cercando di far sì che il discriminatore assegni una probabilità alta (vicina a 1) ai dati generati. Aggiorna i suoi pesi per alzare $D(G(z))$, cercando di far sembrare i dati generati più realistici.

Ci sono delle divergenze che mi permettono di capire quando sta avvenendo l'equilibrio di Nash.

Divergenza di Kullback-Leibler (KL) La divergenza di Kullback-Leibler (KL), nota anche come entropia relativa, misura la differenza tra due distribuzioni di probabilità. Nel contesto delle GAN, possiamo usare la divergenza KL per misurare quanto la distribuzione dei dati generati dal generatore si discosta dalla distribuzione dei dati reali.

Per due distribuzioni di probabilità discrete P e Q , la divergenza KL è definita come:

$$D_{KL}(P||Q) = \sum_x P(x) \log \left(\frac{P(x)}{Q(x)} \right)$$

Dove:

- $P(x)$ è la probabilità del dato x nella distribuzione dei dati reali.
- $Q(x)$ è la probabilità del dato x nella distribuzione dei dati generati dal generatore.
- La somma è calcolata su tutti i possibili dati x .
- $||$ è l'operatore di confronto tra le due distribuzioni, indicando che stiamo misurando la divergenza di P rispetto a Q .

La divergenza KL è sempre non negativa e misura quanto la distribuzione Q si discosta da P . Se Q è identica a P , allora $D_{KL}(P||Q) = 0$, indicando che le due distribuzioni sono identiche. Se Q si discosta significativamente da P , allora $D_{KL}(P||Q)$ sarà maggiore di zero, indicando una maggiore differenza tra le due distribuzioni.

Divergenza di Jensen-Shannon (JSD) La divergenza di Jensen-Shannon (JS) è una misura simmetrica della differenza tra due distribuzioni di probabilità. Nel contesto delle GAN, la divergenza JS viene spesso utilizzata per valutare quanto la distribuzione dei dati generati dal generatore si discosta dalla distribuzione dei dati reali.

Per due distribuzioni di probabilità discrete P e Q , la divergenza JS è definita come:

$$D_{JS}(P||Q) = \frac{1}{2}D_{KL}(P||M) + \frac{1}{2}D_{KL}(Q||M)$$

Dove:

- $P(x)$ è la probabilità del dato x nella distribuzione dei dati reali.
- $Q(x)$ è la probabilità del dato x nella distribuzione dei dati generati dal generatore.
- $M(x) = \frac{1}{2}(P(x) + Q(x))$ è la media delle due distribuzioni.
- $D_{KL}(P||M)$ è la divergenza KL tra P e M , mentre $D_{KL}(Q||M)$ è la divergenza KL tra Q e M .
- $||$ è l'operatore di confronto tra le due distribuzioni, indicando che stiamo misurando la divergenza di P rispetto a Q .
- La divergenza JS è sempre non negativa e simmetrica, il che significa che $D_{JS}(P||Q) = D_{JS}(Q||P)$. Se P e Q sono identiche, allora $D_{JS}(P||Q) = 0$, indicando che le due distribuzioni sono identiche. Se P e Q si discostano significativamente, allora $D_{JS}(P||Q)$ sarà maggiore di zero, indicando una maggiore differenza tra le due distribuzioni.

In parole semplici, il discriminatore cerca di massimizzare la distanza tra le due distribuzioni, mentre il generatore, minimizzando la funzione costo, sta cercando di abbassare la JSD fino a 0. Quando $JSD=0$, significa che p_{data} e p_g sono identiche, quindi il generatore ha raggiunto il suo obiettivo di creare dati che sono indistinguibili dai dati reali, e il discriminatore non può più distinguere tra i due, raggiungendo così un equilibrio di Nash.

Il JSD soffre del problema "gradiente svanente" se le distribuzioni sono troppo distanti, il che significa che il generatore non riceve feedback utili per migliorare la sua capacità di generare dati realistici.

Per risolvere questo problema, sono state proposte varianti delle GAN, come le Wasserstein GAN (WGAN), che utilizzano una diversa misura di distanza tra le distribuzioni, chiamata distanza di Wasserstein, che fornisce un gradiente più stabile anche quando le distribuzioni sono distanti.

La Earth Mover's Distance (EMD), nota anche come distanza di Wasserstein, è una misura di distanza tra due distribuzioni di probabilità. Nel contesto delle GAN, la distanza di Wasserstein viene utilizzata per valutare quanto la distribuzione dei dati generati dal generatore si discosta dalla distribuzione dei dati reali, fornendo un gradiente più stabile durante l'addestramento.

C'è un piccolo inghippo: calcolare esattamente la distanza di Wasserstein può essere computazionalmente costoso, quindi i matematici usano un trucco chiamato "dualità di Kantorovich-Rubinstein" per riformulare il problema in modo che sia più efficiente da risolvere.

In pratica, invece di calcolare direttamente la distanza di Wasserstein, si addestra un "critico" (una rete neurale) che cerca di massimizzare la differenza

tra le aspettative dei dati reali e dei dati generati, soggetta a una condizione di Lipschitz.

Il critico cerca di massimizzare la differenza tra l'aspettativa del logaritmo della probabilità che assegna ai dati reali e l'aspettativa del logaritmo della probabilità che assegna ai dati generati, mentre il generatore cerca di minimizzare questa differenza.

In questo modo, le WGAN forniscono un gradiente più stabile durante l'addestramento, anche quando le distribuzioni dei dati reali e generati sono distanti, consentendo al generatore di migliorare gradualmente la qualità dei dati generati fino a raggiungere un equilibrio di Nash con il critico.